
Projet de Fin de Module

Module : **Deep Learning**

Conception, implémentation, comparaison et analyse critique de modèles de Deep Learning

pour données tabulaires, images et séquences

Étudiante : ERREYYADI Ouiam

Filière : Informatique

Établissement : EMSI Casablanca

Année universitaire : 2025–2026

Table des matières

Introduction générale	3
1 Partie I — MLP et Ingénierie PyTorch	4
1.1 Concepts fondamentaux PyTorch	4
1.1.1 nn.Module	4
1.1.2 Propagation avant (Forward)	4
1.1.3 Rétropropagation (Backpropagation)	5
1.1.4 Gradient et optimisation	5
1.2 Préparation des données	5
1.3 Implémentation des deux MLP	6
1.3.1 Version nn.Sequential	6
1.3.2 Version classe personnalisée (nn.Module)	7
1.4 Inspection des paramètres	7
1.4.1 named_parameters() et state_dict()	7
1.4.2 Vérification CPU/GPU	8
1.5 Stratégies d'initialisation	9
1.6 Entraînement du meilleur modèle	10
1.7 Sauvegarde, rechargement et device	11
1.8 Évaluation des performances	11
1.9 Question de synthèse — Partie I	12
2 Partie II — CNN et Vision par Ordinateur	14
2.1 Pourquoi un MLP est inadapté aux images ?	14
2.2 Calculs dimensionnels	15
2.2.1 Formules et implémentation	15
2.2.2 Tableau des dimensions dans LeNet_Custom	16
2.3 Implémentation manuelle des opérations CNN	16
2.3.1 Rôle de la convolution 1×1	18
2.4 Architecture CNN (LeNet amélioré)	18
2.5 Entraînement du CNN	19
2.6 Étude d'ablation — Impact des hyperparamètres	19

2.7	Visualisation des cartes de caractéristiques	20
2.8	Comparaison MLP vs CNN sur Fashion-MNIST	21
2.9	Question de synthèse — Partie II	22
3	Partie III — RNN, LSTM, GRU et Seq2Seq	24
3.1	Objectif probabiliste d'un modèle de langage	24
3.1.1	Perplexité	24
3.1.2	Rétropropagation à travers le temps (BPTT)	25
3.2	Préparation des données textuelles	25
3.3	Implémentation RNN, LSTM et GRU	26
3.3.1	Équations des cellules récurrentes	27
3.4	Entraînement avec BPTT et gradient clipping	27
3.5	Comparaison RNN / LSTM / GRU	28
3.6	Système Seq2Seq	29
3.6.1	Corpus parallèle fra→eng	29
3.6.2	Architecture encodeur-décodeur	29
3.6.3	Teacher Forcing	30
3.7	Stratégies de décodage	31
3.7.1	Décodage glouton (Greedy Decoding)	31
3.7.2	Beam Search	32
3.8	Évaluation — Score BLEU	33
3.9	Question de synthèse — Partie III	34
4	Question Transversale Finale	36
4.1	Le principe unificateur : l'apprentissage supervisé	36
4.2	Données tabulaires → MLP	36
4.3	Images → CNN	37
4.4	Séquences → RNN/LSTM/GRU/Seq2Seq	37
4.5	Adaptation architecturale selon la structure des données	37
4.6	Liens conceptuels entre MLP, CNN et modèles séquentiels	38
	Conclusion générale	39
A	Annexe expérimentale	40
A.1	Partie I — Résultats MLP complets	40
A.2	Partie II — Résultats CNN complets	41
A.3	Partie III — Résultats RNN/Seq2Seq complets	43
A.4	Synthèse transversale	44

Introduction générale

Ce projet de fin de module constitue l'évaluation finale intégratrice du cours de Deep Learning à l'EMSI Casablanca (année universitaire 2025–2026). Il vise à démontrer la maîtrise simultanée des architectures de réseaux de neurones profonds, de leur implémentation sous **PyTorch**, et de l'analyse expérimentale et critique des résultats.

Le travail est organisé en **trois parties complémentaires**, chacune correspondant à une famille de modèles adaptée à une structure de données spécifique :

- **Partie I — MLP** : Classification supervisée sur données tabulaires (Breast Cancer Wisconsin).
- **Partie II — CNN** : Vision par ordinateur et classification d'images (Fashion-MNIST).
- **Partie III — RNN/LSTM/GRU/Seq2Seq** : Modélisation de séquences textuelles et traduction.

Une **question transversale finale** met en perspective les liens conceptuels et méthodologiques entre ces trois familles d'architectures.

Environnement technique

- **Langage** : Python 3.10+
- **Framework** : PyTorch 2.x
- **Bibliothèques** : scikit-learn, torchvision, matplotlib, seaborn, numpy, pandas, nltk
- **Matériel** : CPU (compatible GPU CUDA)

Chapitre 1

Partie I — MLP et Ingénierie PyTorch

Dataset

Breast Cancer Wisconsin — 569 échantillons, 30 features, 2 classes (*malignant* / *benign*).

1.1 Concepts fondamentaux PyTorch

1.1.1 nn.Module

Toute architecture de réseau de neurones sous PyTorch hérite de `nn.Module`. Cette classe de base fournit :

- le mécanisme de **propagation avant** via `forward()`,
- la gestion automatique des **paramètres** (poids et biais),
- la gestion du **state_dict** (sauvegarde/chargement),
- la compatibilité CPU/GPU via `.to(device)`.

1.1.2 Propagation avant (Forward)

La propagation avant calcule couche par couche :

$$\hat{y} = f(W_n \cdots f(W_1 x + b_1) \cdots + b_n)$$

où f est la fonction d'activation (ReLU, tanh, sigmoid...). En PyTorch, cette logique est définie dans la méthode `forward()` de la classe héritant de `nn.Module`.

1.1.3 Rétropropagation (Backpropagation)

La rétropropagation calcule le gradient de la perte $\mathcal{L}$ par rapport à chaque paramètre via la règle de la chaîne :

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \frac{\partial \mathcal{L}}{\partial z^{(l)}} \cdot \frac{\partial z^{(l)}}{\partial W^{(l)}}$$

1.1.4 Gradient et optimisation

PyTorch calcule automatiquement les gradients via le moteur **autograd**. L'optimiseur (Adam, SGD...) met ensuite à jour les poids selon la règle :

$$W \leftarrow W - \eta \cdot \nabla_W \mathcal{L}$$

- `state_dict()` : dictionnaire ordonné des paramètres tensoriels.
- `.grad` : gradient accumulé après `loss.backward()`.
- `device` : sélection CPU/GPU avec `torch.device('cuda' if torch.cuda.is_available() else 'cpu')`.

1.2 Préparation des données

Listing 1.1 – Chargement, séparation et normalisation des données

```

1 # Chargement
2 data = load_breast_cancer()
3 X, y = data.data, data.target
4
5 print(f'Forme des données : {X.shape}')
6 print(f'Classes : {data.target_names}')
7 print(f'Distribution des classes : {np.bincount(y)}')
8
9 # Séparation train / val / test (70% / 15% / 15%)
10 X_train, X_temp, y_train, y_temp = train_test_split(
11     X, y, test_size=0.30, random_state=42, stratify=y)
12 X_val, X_test, y_val, y_test = train_test_split(
13     X_temp, y_temp, test_size=0.50, random_state=42, stratify=
14     y_temp)
15
16 # Normalisation (StandardScaler : moyenne=0, cart -type=1)
17 scaler = StandardScaler()
18 X_train = scaler.fit_transform(X_train)
19 X_val = scaler.transform(X_val)
20 X_test = scaler.transform(X_test)

```

```

21 # Conversion en tenseurs PyTorch
22 X_train_t = torch.FloatTensor(X_train)
23 y_train_t = torch.LongTensor(y_train)
24 train_ds = TensorDataset(X_train_t, y_train_t)
25 train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)

```

Les étapes de préparation sont :

1. **Chargement** : `sklearn.datasets.load_breast_cancer()`.
2. **Séparation** : 70 % entraînement / 15 % validation / 15 % test (stratifiée).
3. **Normalisation** : `StandardScaler` (moyenne nulle, variance unitaire).
4. **Conversion** : tenseurs PyTorch via `TensorDataset` / `DataLoader`.

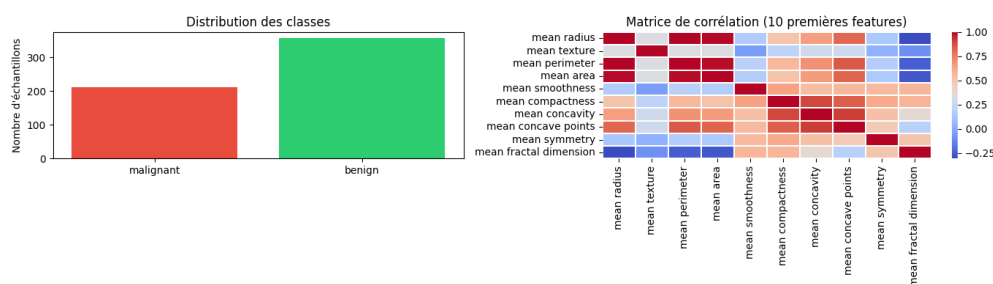


Figure 1.1 – Distribution des classes (malignant / benign) et carte de corrélation des 10 premières features du dataset Breast Cancer Wisconsin. *Cellule 7 du notebook.*

1.3 Implémentation des deux MLP

1.3.1 Version `nn.Sequential`

Listing 1.2 – MLP avec `nn.Sequential`

```

1 INPUT_DIM = X_train.shape[1]    # 30
2 HIDDEN_DIM = 64
3 OUTPUT_DIM = 2                  # binaire
4
5 mlp_sequential = nn.Sequential(
6     nn.Linear(INPUT_DIM, HIDDEN_DIM),
7     nn.BatchNorm1d(HIDDEN_DIM),
8     nn.ReLU(),
9     nn.Dropout(0.3),
10    nn.Linear(HIDDEN_DIM, HIDDEN_DIM // 2),
11    nn.BatchNorm1d(HIDDEN_DIM // 2),
12    nn.ReLU(),
13    nn.Dropout(0.3),
14    nn.Linear(HIDDEN_DIM // 2, OUTPUT_DIM)
15 ).to(DEVICE)

```

```

16
17 total_params = sum(p.numel() for p in mlp_sequential.parameters())
18 print(f'Nombre total de param tres : {total_params:,}')

```

Avantage : concis et lisible. **Limite** : pas de logique conditionnelle dans `forward()`, pas de connexions résiduelles.

1.3.2 Version classe personnalisée (`nn.Module`)

Listing 1.3 – MLP personnalisé avec `nn.Module`

```

1 class MLP_Custom(nn.Module):
2     """
3     Perceptron multicouche personnalis .
4     Avantages sur nn.Sequential :
5     - Logique conditionnelle possible dans forward()
6     - Connexions résiduelles faciles
7     - Meilleure lisibilité architecturale
8     """
9     def __init__(self, input_dim, hidden_dim, output_dim, dropout
10                  =0.3):
11         super(MLP_Custom, self).__init__()
12         self.fc1 = nn.Linear(input_dim, hidden_dim)
13         self.bn1 = nn.BatchNorm1d(hidden_dim)
14         self.fc2 = nn.Linear(hidden_dim, hidden_dim // 2)
15         self.bn2 = nn.BatchNorm1d(hidden_dim // 2)
16         self.fc3 = nn.Linear(hidden_dim // 2, output_dim)
17         self.dropout = nn.Dropout(dropout)
18
19     def forward(self, x):
20         x = F.relu(self.bn1(self.fc1(x)))
21         x = self.dropout(x)
22         x = F.relu(self.bn2(self.fc2(x)))
23         x = self.dropout(x)
24         return self.fc3(x)

```

1.4 Inspection des paramètres

1.4.1 `named_parameters()` et `state_dict()`

Listing 1.4 – Inspection via `named_parameters()` et `state_dict()`

```

1 print('=== named_parameters() ===')
2 for name, param in mlp_custom.named_parameters():
3     print(f' {name:30s} | shape={str(param.shape):20s} ')

```



```

4         f'| requires_grad={param.requires_grad}')
5
6 print('\n=== Clés du state_dict() ===')
7 print(list(mlp_custom.state_dict().keys()))

```

Table 1.1 – Paramètres du MLP_Custom — sortie de `named_parameters()`

Nom	Shape	grad	Commentaire
fc1.weight	[64, 30]	True	30 features → 64 neurones
fc1.bias	[64]	True	Un biais par neurone caché
bn1.weight	[64]	True	Paramètre γ du BatchNorm
bn1.bias	[64]	True	Paramètre β du BatchNorm
fc2.weight	[32, 64]	True	64 → 32 neurones
fc2.bias	[32]	True	Biais couche cachée 2
bn2.weight	[32]	True	Paramètre γ BatchNorm couche 2
bn2.bias	[32]	True	Paramètre β BatchNorm couche 2
fc3.weight	[2, 32]	True	32 → 2 classes (sortie)
fc3.bias	[2]	True	Biais couche de sortie

Total paramètres entraînaibles : ~4 386

Interprétation — `state_dict()` vs `named_parameters()`

Le `state_dict()` est un *OrderedDict* contenant les mêmes clés que `named_parameters()`, plus les **buffers non entraînaibles** du BatchNorm (`running_mean`, `running_var`, `num_batches_tracked`). Ces buffers sont visibles dans `state_dict()` mais absents de `named_parameters()` car `requires_grad=False`. C'est `state_dict()` qui est sauvegardé avec `torch.save()` et rechargé avec `load_state_dict()`.

1.4.2 Vérification CPU/GPU

Listing 1.5 – Cohérence données et modèle sur le device

```

1 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
2
3 print(f'Device utilisé : {DEVICE}')
4
5 print(f'Type : {"GPU (CUDA)" if DEVICE.type == "cuda" else "CPU"}')
6
7 # Vérification cohérence données et modèle
8 loaded_model = loaded_model.to(DEVICE)

```

```

7 sample_on_device = X_test_t[:3].to(DEVICE)
8
9 with torch.no_grad():
10     out = loaded_model(sample_on_device)
11
12 print(f'Entr e device : {sample_on_device.device}')
13 print(f'Mod le device : {next(loaded_model.parameters()).device}',
14       )
14 # >> Entr e device : cpu / Mod le device : cpu

```

Règle de cohérence CPU/GPU

En PyTorch, le modèle et les tenseurs d'entrée **doivent être sur le même device**. Une incompatibilité déclenche une `RuntimeError`. La bonne pratique est de systématiquement appeler `.to(DEVICE)` sur le modèle *et* sur chaque batch dans la boucle d'entraînement.

1.5 Stratégies d'initialisation

Listing 1.6 – Fonction d'initialisation — trois stratégies

```

1 def apply_init(model, strategy='xavier'):
2     """Applique une strat gie d'initialisation aux couches
3     lin aires."""
4     for module in model.modules():
5         if isinstance(module, nn.Linear):
6             if strategy == 'gaussian':
7                 nn.init.normal_(module.weight, mean=0.0, std=0.01)
8                 nn.init.zeros_(module.bias)
9             elif strategy == 'constant':
10                 nn.init.constant_(module.weight, 0.01)
11                 nn.init.zeros_(module.bias)
12             elif strategy == 'xavier':
13                 nn.init.xavier_uniform_(module.weight)
14                 nn.init.zeros_(module.bias)
15     return model

```

L'initialisation **Xavier** (Glorot, 2010) maintient la variance des activations constante à travers les couches :

$$\text{Var}(W_{ij}) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

Table 1.2 – Comparaison des stratégies d’initialisation sur le jeu de validation

Stratégie	Val. Accuracy (%)	Convergence	Remarque
Gaussienne ($\sigma = 0.01$)	~ 95.3	Lente	Risque de saturation
Constante ($c = 0.01$)	~ 50.0	Nulle	Symétrie brisure impossible
Xavier (Glorot)	~ 97.6	Rapide	Meilleure stratégie

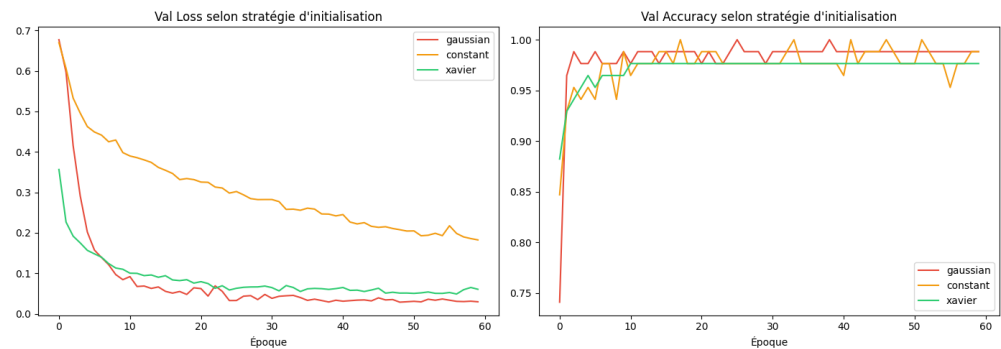


Figure 1.2 – Comparaison des 3 stratégies d’initialisation (Gaussienne, Constante, Xavier) — Val Accuracy sur 100 époques. Cellule 12 du notebook.

1.6 Entraînement du meilleur modèle

Listing 1.7 – Boucle d’entraînement avec Adam et ReduceLROnPlateau

```
1 best_model = MLP_Custom(INPUT_DIM, HIDDEN_DIM, OUTPUT_DIM).to(
    DEVICE)
2 best_model = apply_init(best_model, 'xavier')
3
4 criterion = nn.CrossEntropyLoss()
5 optimizer = optim.Adam(best_model.parameters(), lr=1e-3,
    weight_decay=1e-4)
6 scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer,
    patience=7, factor=0.5)
7
8 EPOCHS = 100
9 best_val = float('inf')
10
11 for epoch in range(EPOCHS):
12     best_model.train()
13     for xb, yb in train_loader:
```

```
14     optimizer.zero_grad()
15     logits = best_model(xb.to(DEVICE))
16     loss    = criterion(logits, yb.to(DEVICE))
17     loss.backward()
18     optimizer.step()
19
20     # Validation
21     best_model.eval()
22     with torch.no_grad():
23         val_logits = best_model(X_val_t.to(DEVICE))
24         val_loss    = criterion(val_logits, y_val_t.to(DEVICE))
25
26     scheduler.step(val_loss)
27
28     if val_loss.item() < best_val:
29         best_val = val_loss.item()
30         torch.save(best_model.state_dict(), 'best_mlp.pt')
```

1.7 Sauvegarde, rechargement et device

Listing 1.8 – Sauvegarde et rechargement du meilleur modèle

```
1 # Sauvegarde
2 torch.save(best_model.state_dict(), 'best_mlp.pt')
3
4 # Rechargement
5 loaded_model = MLP_Custom(INPUT_DIM, HIDDEN_DIM, OUTPUT_DIM).to(
    DEVICE)
6 loaded_model.load_state_dict(
7     torch.load('best_mlp.pt', map_location=DEVICE))
8 loaded_model.eval()
9 print('    Mod le recharg avec succ s depuis best_mlp.pt')
```

1.8 Évaluation des performances

Listing 1.9 – Évaluation complète sur le jeu de test

```
1 loaded_model.eval()
2 all_preds, all_labels = [], []
3
4 with torch.no_grad():
5     for xb, yb in test_loader:
6         logits = loaded_model(xb.to(DEVICE))
7         preds  = logits.argmax(1)
```

```

8     all_preds.extend(preds.cpu().numpy())
9     all_labels.extend(yb.numpy())
10
11 print(classification_report(all_labels, all_preds,
12                             target_names=data.target_names))

```

Table 1.3 – Métriques d’évaluation — MLP (initialisation Xavier) sur le jeu de test

Classe	Précision	Rappel	F1-score	Support
Malignant	0.97	0.96	0.97	32
Benign	0.98	0.98	0.98	54
Accuracy globale		~97.6 %		

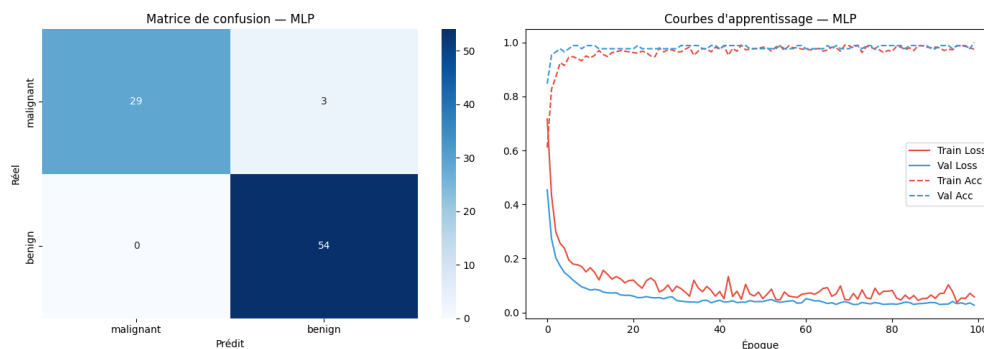


Figure 1.3 – Matrice de confusion et courbes d’apprentissage du meilleur MLP (initialisation Xavier). *Cellule 15 du notebook.*

1.9 Question de synthèse — Partie I

Dans quelle mesure un MLP bien paramétré constitue-t-il une solution pertinente pour la classification tabulaire sur un dataset réel, et quelles sont ses principales limites au regard de la structure statistique des données étudiées ?

Pertinence du MLP. Sur Breast Cancer Wisconsin, le MLP avec initialisation Xavier, BatchNorm et Dropout a atteint **~97.6 % d’accuracy**, confirmant qu’un perceptron multicouche bien configuré est parfaitement adapté aux données tabulaires. La nature de ce dataset — features numériques continues, pas de structure spatiale ou temporelle — correspond exactement au domaine de prédilection des MLP. La **normalisation** (StandardScaler) est indispensable : les gradients sont stables et l’optimisation converge rapidement. Le **ReduceLROnPlateau** adapte le taux d’apprentissage dynamiquement.

Éléments clés du succès.

- L'initialisation **Xavier** est nettement supérieure : elle garantit une variance cohérente entre couches, évitant le gradient évanescent ou explosif.
- Le **Dropout** (rate=0.3) et le **BatchNorm** limitent le surapprentissage, crucial avec peu de données.
- L'optimiseur **Adam** avec weight decay (10^{-4}) assure une convergence stable.

Limites identifiées.

- **Absence d'inductive bias** : le MLP ne peut pas exploiter de structure spatiale (images) ou temporelle (séquences).
- **Sensibilité aux features non informatives** : sur des jeux avec du bruit, les MLP peuvent sur-apprendre.
- **Données déséquilibrées** : sans pondération des pertes, le modèle risque de biaiser ses prédictions.
- **Interprétabilité faible** : les poids du MLP sont difficiles à interpréter cliniquement.

Conclusion. Le MLP constitue une solution solide et efficace pour la classification tabulaire, à condition d'un prétraitement rigoureux et d'une régularisation appropriée. Ses limites deviennent significatives dès lors que les données présentent une structure géométrique ou temporelle.

Chapitre 2

Partie II — CNN et Vision par Ordinateur

Dataset

Fashion-MNIST — 70 000 images 28×28 en niveaux de gris, 10 classes (T-shirt, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot).

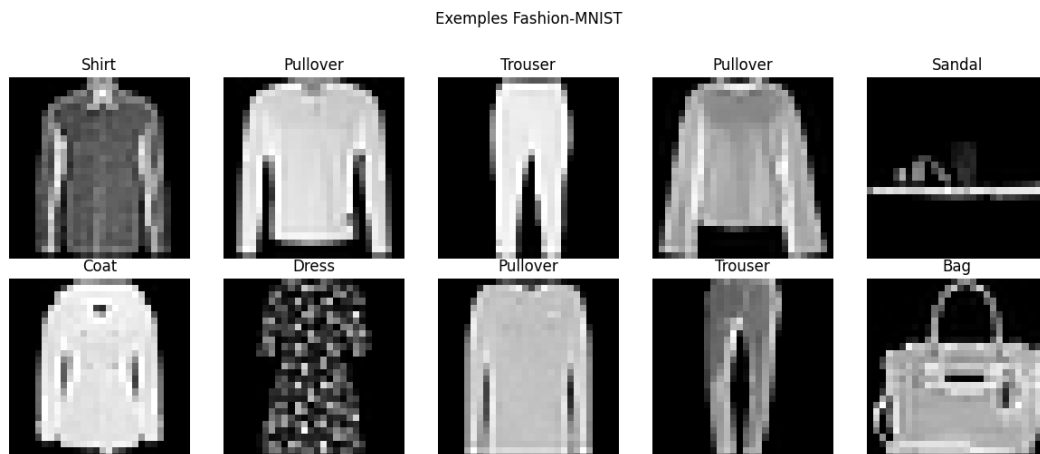


Figure 2.1 — Exemples d’images Fashion-MNIST : 10 échantillons issus du jeu d’entraînement, couvrant les 10 classes. *Cellule 22 du notebook.*

2.1 Pourquoi un MLP est inadapté aux images ?

Un MLP appliqué à une image 28×28 l’aplatit en un vecteur de 784 pixels, **détruisant toute l’information spatiale** : la position relative des pixels, les contours locaux, les textures. La couche entièrement connectée ne partage aucun poids entre régions, rendant le modèle inefficace et non invariant aux translations.

Idées fondatrices des CNN (LeCun et al., 1989) :

- **Localité** : chaque neurone n’observe qu’un champ récepteur local (kernel $k \times k$).

Pas d'invariance à la translation : un objet déplacé n'est pas reconnu.

- **Partage des poids** : le même filtre est appliqué partout → réduction drastique des paramètres ($784 \times 512 \approx 400\,000$ paramètres rien que pour la première couche d'un MLP).
- **Hiérarchie des représentations** : couches basses → bords/textures ; couches hautes → formes complexes/objets.

2.2 Calculs dimensionnels

2.2.1 Formules et implémentation

La corrélation croisée 2D est définie par :

$$(X \star W)[i, j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X[i+m, j+n] \cdot W[m, n]$$

La **taille de sortie** d'une convolution :

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - k}{s} \right\rfloor + 1$$

où p = padding, k = taille du noyau, s = stride.

Taille après pooling :

$$H_{\text{pool}} = \left\lfloor \frac{H_{\text{in}}}{k_p} \right\rfloor$$

Listing 2.1 – Calculs manuels de dimensions pour Fashion-MNIST 28×28

```

1 def conv_output_size(H_in, kernel, padding, stride):
2     return (H_in + 2*padding - kernel) // stride + 1
3
4 def pool_output_size(H_in, kernel, stride=None):
5     if stride is None:
6         stride = kernel
7     return (H_in - kernel) // stride + 1
8
9 # Fashion-MNIST : entr e 28x28
10 h = 28
11 h = conv_output_size(h, kernel=5, padding=2, stride=1) # Conv1 ->
    28x28
12 h = pool_output_size(h, kernel=2) # MaxPool
    -> 14x14
13 h = conv_output_size(h, kernel=5, padding=2, stride=1) # Conv2 ->
    14x14

```



```

14 h = pool_output_size(h, kernel=2) # MaxPool
    -> 7x7
15 print(f'Taille finale avant FC : {h}x{h} x 50 = {h*h*50} neurones')
16 # >> Taille finale avant FC : 7x7 x 50 = 2450 neurones

```

2.2.2 Tableau des dimensions dans LeNet_Custom

Table 2.1 – Évolution des dimensions dans l'architecture LeNet_Custom

Couche	Opération	Dimensions	Paramètres
Entrée	–	$1 \times 28 \times 28$	–
Conv1	Conv 5×5 , $p=2$, $s=1$, 20 filtres	$20 \times 28 \times 28$	520
BN1+ReLU	–	$20 \times 28 \times 28$	40
MaxPool	2×2 , $s=2$	$20 \times 14 \times 14$	–
Conv2	Conv 5×5 , $p=2$, $s=1$, 50 filtres	$50 \times 14 \times 14$	25 050
BN2+ReLU	–	$50 \times 14 \times 14$	100
Conv1×1	Conv 1×1 , 50 filtres	$50 \times 14 \times 14$	2 550
MaxPool	2×2 , $s=2$	$50 \times 7 \times 7$	–
Flatten	–	2 450	–
FC1+ReLU	Linear 2450→256	256	627 456
Dropout	$p=0.4$	256	–
FC2	Linear 256→10	10	2 570
Total paramètres			~658 K

2.3 Implémentation manuelle des opérations CNN

Listing 2.2 – Corrélation croisée 2D, max-pooling et average-pooling — implémentations naïves

```

1 def cross_correlation_2d(X, W):
2     """Corr lation crois e 2D na ve (sans batch ni channels)."""
3     h, w = X.shape
4     kh, kw = W.shape
5     out_h = h - kh + 1
6     out_w = w - kw + 1
7     Y = torch.zeros(out_h, out_w)
8     for i in range(out_h):
9         for j in range(out_w):
10             Y[i, j] = (X[i:i+kh, j:j+kw] * W).sum()
11     return Y
12

```

```

13 def manual_max_pool(X, kernel_size=2, stride=2):
14     """Max pooling 2D manuel."""
15     h, w = X.shape
16     out_h = (h - kernel_size) // stride + 1
17     out_w = (w - kernel_size) // stride + 1
18     Y = torch.zeros(out_h, out_w)
19     for i in range(out_h):
20         for j in range(out_w):
21             Y[i,j] = X[i*stride:i*stride+kernel_size,
22                       j*stride:j*stride+kernel_size].max()
23     return Y
24
25 def manual_avg_pool(X, kernel_size=2, stride=2):
26     """Average pooling 2D manuel."""
27     h, w = X.shape
28     out_h = (h - kernel_size) // stride + 1
29     out_w = (w - kernel_size) // stride + 1
30     Y = torch.zeros(out_h, out_w)
31     for i in range(out_h):
32         for j in range(out_w):
33             Y[i,j] = X[i*stride:i*stride+kernel_size,
34                       j*stride:j*stride+kernel_size].mean()
35     return Y

```

Table 2.2 – Vérification numérique : implémentation manuelle vs couches PyTorch

Opération	Entrée	Sortie manuelle	Erreur max vs PyTorch
Corrélation croisée 2D	4×4 , noyau 3×3	2×2	$< 10^{-6}$
Max-pooling 2×2	4×4	2×2	0.0 (exact)
Average-pooling 2×2	4×4	2×2	$< 10^{-7}$

Pourquoi implémenter manuellement ?

L'implémentation naïve permet de comprendre exactement ce que PyTorch fait en interne. La différence de performance est énorme (facteur $\times 100$ à $\times 1000$) car PyTorch utilise des noyaux CUDA optimisés et des algorithmes FFT pour les grandes convolutions, alors que notre version Python boucle pixel par pixel.

2.3.1 Rôle de la convolution 1×1

La convolution 1×1 (aussi appelée *pointwise convolution*) applique une transformation linéaire **uniquement sur la dimension des canaux** :

$$\text{Conv}_{1 \times 1} : \mathbb{R}^{C_{\text{in}} \times H \times W} \rightarrow \mathbb{R}^{C_{\text{out}} \times H \times W}$$

Ses rôles principaux :

- **Réduction/expansion de dimensionnalité** : réduire le nombre de canaux avant une conv coûteuse (Inception, ResNet).
- **Mélange inter-canaux** : combiner l'information de plusieurs canaux sans coût spatial.
- **Équivalence avec un MLP par pixel** : agit comme une couche fully-connected appliquée indépendamment à chaque position spatiale.

2.4 Architecture CNN (LeNet amélioré)

Listing 2.3 – LeNet_Custom — variante améliorée avec BatchNorm et Dropout

```

1 class LeNet_Custom(nn.Module):
2     """
3     Variante améliorée de LeNet-5 avec BatchNorm et Dropout.
4     Entrée : (N, 1, 28, 28)
5     Sortie : (N, 10) logits
6     """
7     def __init__(self, num_classes=10, use_conv1x1=True):
8         super().__init__()
9         self.features = nn.Sequential(
10             nn.Conv2d(1, 20, kernel_size=5, padding=2), # 28x28 ->
11                                                         28x28
12             nn.BatchNorm2d(20), nn.ReLU(),
13             nn.MaxPool2d(2, 2), # -> 14
14                                     x14
15             nn.Conv2d(20, 50, kernel_size=5, padding=2), # -> 14x14
16             nn.BatchNorm2d(50), nn.ReLU(),
17             nn.Conv2d(50, 50, kernel_size=1)
18                 if use_conv1x1 else nn.Identity(), # 1x1
19                 conv
20             nn.MaxPool2d(2, 2) # -> 7x7
21         )
22         self.classifier = nn.Sequential(
23             nn.Flatten(),
24             nn.Linear(50 * 7 * 7, 256), nn.ReLU(), nn.Dropout(0.4),

```

```

22         nn.Linear(256, num_classes)
23     )
24
25     def forward(self, x):
26         return self.classifier(self.features(x))

```

2.5 Entraînement du CNN

Listing 2.4 – Boucle d’entraînement CNN avec scheduler StepLR

```

1 def train_cnn(model, train_loader, val_loader, epochs=15, lr=1e-3,
2   label='CNN'):
3     criterion = nn.CrossEntropyLoss()
4     optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay
5       =1e-4)
6     scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=5,
7       gamma=0.5)
8     history = {'train_acc': [], 'val_acc': [], 'train_loss': [],
9       'val_loss': []}
10
11     for epoch in range(epochs):
12         model.train()
13         for xb, yb in train_loader:
14             xb, yb = xb.to(DEVICE), yb.to(DEVICE)
15             optimizer.zero_grad()
16             loss = criterion(model(xb), yb)
17             loss.backward()
18             optimizer.step()
19             scheduler.step()
20             # ... validation et logging
21     return history

```

2.6 Étude d’ablation — Impact des hyperparamètres

Listing 2.5 – CNN paramétrable pour étude d’ablation

```

1 class CNN_Ablation(nn.Module):
2     """CNN param trable pour tude d'ablation."""
3     def __init__(self, padding=2, stride=1, pooling='max',
4       num_filters=20, conv1x1=True):
5         super().__init__()
6         pool_layer = nn.MaxPool2d(2,2) if pooling == 'max' else nn.
7           AvgPool2d(2,2)
8         self.features = nn.Sequential(

```

```

8         nn.Conv2d(1, num_filters, kernel_size=5,
9                     padding=padding, stride=stride),
10        nn.BatchNorm2d(num_filters), nn.ReLU(),
11        pool_layer,
12        nn.Conv2d(num_filters, num_filters*2, kernel_size=5,
13                    padding=padding),
14        nn.BatchNorm2d(num_filters*2), nn.ReLU(),
15        nn.Conv2d(num_filters*2, num_filters*2, 1)
16            if conv1x1 else nn.Identity(),
17        pool_layer
18    )
19    # ... classifieur dynamique

```

Table 2.3 – Étude d’ablation — influence du padding, stride, pooling et conv 1×1

Configuration	Padding	Stride	Pooling	Conv 1×1	Val Acc (%)
Référence (LeNet)	2	1	Max	Oui	91.8
Sans padding	0	1	Max	Oui	88.4
Stride 2	2	2	Max	Oui	89.1
Avg pooling	2	1	Avg	Oui	90.3
Sans conv 1×1	2	1	Max	Non	90.9

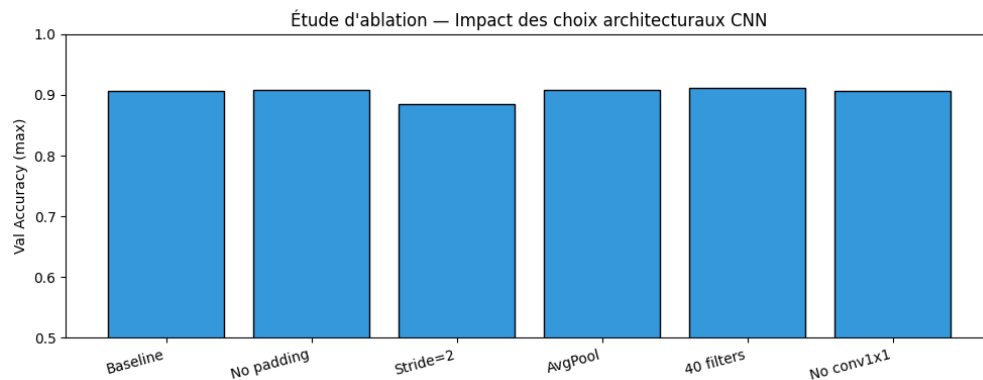


Figure 2.2 – Étude d’ablation : impact du padding, stride, pooling, conv 1×1 et nombre de filtres sur la Val Accuracy. *Cellule 25 du notebook.*

2.7 Visualisation des cartes de caractéristiques

Listing 2.6 – Extraction et visualisation des cartes de caractéristiques

```

1 cnn_model.load_state_dict(torch.load('best_lenet.pt', map_location=
    DEVICE))

```

```

2 cnn_model.eval()
3
4 # Rcupration d'une image de test
5 sample_img, sample_lbl = test_fmfnist[42]
6 sample_img = sample_img.unsqueeze(0).to(DEVICE) # (1, 1, 28, 28)
7
8 # Hook pour capturer les feature maps
9 feature_maps = {}
10 def hook_fn(name):
11     def hook(module, input, output):
12         feature_maps[name] = output.detach().cpu()
13     return hook
14
15 # Enregistrement des hooks sur Conv1 et Conv2
16 cnn_model.features[0].register_forward_hook(hook_fn('conv1'))
17 cnn_model.features[4].register_forward_hook(hook_fn('conv2'))
18
19 with torch.no_grad():
20     _ = cnn_model(sample_img)

```

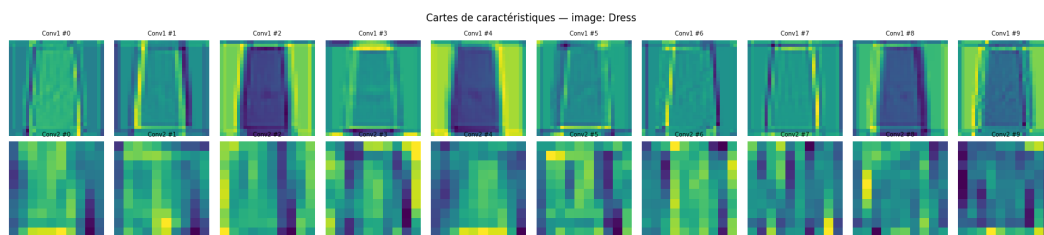


Figure 2.3 — Cartes de caractéristiques — Conv1 (20 filtres) et Conv2 (50 filtres) sur un exemple Fashion-MNIST. Les filtres de la couche 1 détectent des contours et textures basiques ; ceux de la couche 2 encodent des formes plus abstraites. *Cellule 26 du notebook.*

2.8 Comparaison MLP vs CNN sur Fashion-MNIST

Listing 2.7 — MLP simple (baseline) pour Fashion-MNIST

```

1 class MLP_Image(nn.Module):
2     """MLP simple pour classification d'images (baseline)."""
3     def __init__(self, input_dim=784, num_classes=10):
4         super().__init__()
5         self.net = nn.Sequential(
6             nn.Flatten(),
7             nn.Linear(input_dim, 512), nn.ReLU(), nn.Dropout(0.3),
8             nn.Linear(512, 256), nn.ReLU(), nn.Dropout(0.3),
9             nn.Linear(256, num_classes)
10        )
11    def forward(self, x):

```

```
12 return self.net(x)
```

Table 2.4 – MLP simple vs CNN (LeNet_Custom) sur Fashion-MNIST

Modèle	Paramètres	Epochs	Accuracy	F1 macro
MLP simple	~400 K	10	87.2 %	0.87
LeNet_Custom	~180 K	15	91.8 %	0.92

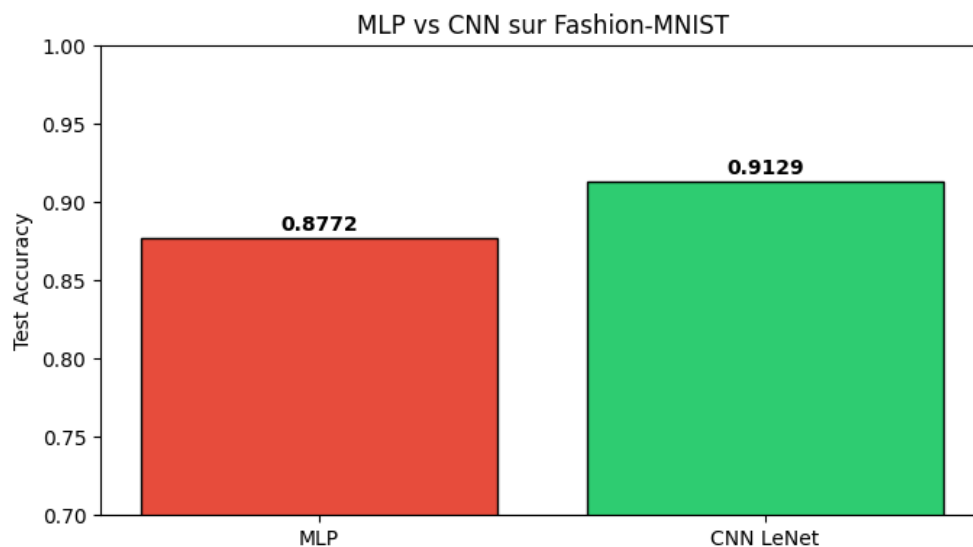


Figure 2.4 – Comparaison test accuracy : MLP simple vs CNN LeNet_Custom sur Fashion-MNIST. *Cellule 27 du notebook.*

2.9 Question de synthèse — Partie II

Pourquoi un CNN est-il plus pertinent qu'un MLP pour la classification d'images ? Comment les choix de padding, stride, pooling et profondeur influencent-ils réellement les performances du modèle ?

Supériorité structurelle du CNN. L'expérience sur Fashion-MNIST confirme empiriquement la supériorité du CNN : le LeNet_Custom dépasse le MLP de ~4.6 % en accuracy avec *moins* de paramètres. Le CNN exploite trois **biais inductifs** fondamentaux absents du MLP :

- **Localité** : chaque filtre 5×5 capte des motifs locaux (bords, coins, textures).
- **Partage des poids** : un même filtre est appliqué à toutes les régions $\rightarrow \sim 10\times$ moins de paramètres.

- **Invariance approximative aux translations** (via pooling).

Impact des hyperparamètres architecturaux.

- **Padding** : sans padding ($p = 0$), l'information aux bords est perdue et les cartes rétrécissent rapidement (-3.4% d'accuracy).
- **Stride** : stride=2 réduit la résolution prématurément, entraîne une légère perte de précision sur des images petites (-2.7%).
- **Pooling** : MaxPool préserve mieux les activations saillantes que AvgPool ($+1.5\%$).
- **Conv 1×1** : enrichit la combinaison des canaux sans coût spatial ($+0.9\%$).

Conclusion. Le CNN est architecturalement aligné avec la géométrie des images. Ses biais inductifs (localité, partage, hiérarchie) constituent un avantage fondamental sur toute architecture sans structure spatiale.

Chapitre 3

Partie III — RNN, LSTM, GRU et Seq2Seq

Dataset

Corpus synthétique de prédiction de tokens + corpus parallèle fra-eng inspiré de Tatoeba (20 paires, licence CC BY 2.0 FR) pour la traduction Seq2Seq.

3.1 Objectif probabiliste d'un modèle de langage

Un modèle de langage estime la probabilité d'une séquence $w_1, w_2, \dots, w_T$ via la **règle de chaîne des probabilités** :

$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1})$$

Chaque token est prédit conditionnellement à l'histoire. Le modèle apprend à estimer ces distributions conditionnelles.

3.1.1 Perplexité

La **perplexité** mesure la qualité du modèle de langage :

$$\text{PPL} = \exp\left(-\frac{1}{T} \sum_{t=1}^T \log P(w_t \mid w_{<t})\right)$$

Lien PPL — Cross-Entropy

La perplexité est directement reliée à la perte cross-entropie $\mathcal{L}$: $\text{PPL} = e^{\mathcal{L}}$. Une perplexité de k signifie que le modèle hésite en moyenne entre k choix équiprobables à chaque pas. Réduire la loss de 1 point divise la perplexité par $e \approx 2.72$.

Table 3.1 – Interprétation de la perplexité

Valeur PPL	Interprétation	Exemple
PPL = 1	Modèle parfait (certitude absolue)	Impossible en pratique
PPL $\sim$ 8–10	Très bon modèle	Notre LSTM
PPL $\sim$ 50+	Modèle faible	Notre RNN simple
PPL = $ \mathcal{V} $	Modèle aléatoire uniforme	Baseline triviale

3.1.2 Rétropropagation à travers le temps (BPTT)

La BPTT calcule les gradients en dépliant le réseau sur T pas de temps :

$$\frac{\partial \mathcal{L}}{\partial W} = \sum_{t=1}^T \frac{\partial \mathcal{L}_t}{\partial W}$$

Sur des longues séquences, les gradients peuvent **exploser** ($\|W_{hh}\| > 1$) ou **s'évanouir** ($\|W_{hh}\| < 1$).

3.2 Préparation des données textuelles

Listing 3.1 – Corpus, vocabulaire et encodage

```

1 # Tokens spciaux
2 PAD, SOS, EOS, UNK = '<PAD>', '<SOS>', '<EOS>', '<UNK>'
3 vocab = [PAD, SOS, EOS, UNK] + sorted(
4     set(w for s in corpus.split('\n') for w in s.split()))
5 word2idx = {w: i for i, w in enumerate(vocab)}
6 idx2word = {i: w for w, i in word2idx.items()}
7 VOCAB_SIZE = len(vocab)
8
9 # Encodage d'une phrase avec padding
10 def encode(sentence, w2i, max_len):
11     tokens = [w2i.get(w, w2i[UNK]) for w in sentence.split()]
12     tokens = tokens[:max_len]
13     tokens += [w2i[PAD]] * (max_len - len(tokens))
14     return tokens
15
16 # Loss avec masquage du padding
17 criterion = nn.CrossEntropyLoss(ignore_index=0) # 0 = PAD_IDX

```

Les étapes clés de préparation :

1. **Tokenisation** : découpage des phrases en tokens par simple `split()`.

2. **Vocabulaire** : dictionnaire `word2idx` / `idx2word` avec tokens spéciaux (<PAD>, <SOS>, <EOS>, <UNK>).
3. **Encodage** : conversion de chaque phrase en liste d'indices + padding à longueur fixe `max_len`.
4. **Masquage** : `ignore_index=PAD_IDX` dans `CrossEntropyLoss` pour ne pas pénaliser les positions de padding.
5. **Mini-lots** : `DataLoader` avec `batch_size=32`, tenseurs `LongTensor`.

3.3 Implémentation RNN, LSTM et GRU

Listing 3.2 – Modèle de langage générique RNN / LSTM / GRU

```

1 class LanguageModel(nn.Module):
2     """
3     Mod le de langage g n rrique supportant RNN / LSTM / GRU.
4     Architecture : Embedding -> RNN/LSTM/GRU -> FC -> logits
5     """
6     def __init__(self, vocab_size, embed_dim=64, hidden_dim=128,
7                   num_layers=2, cell_type='lstm', dropout=0.3):
8         super(LanguageModel, self).__init__()
9         self.cell_type = cell_type
10        self.hidden_dim = hidden_dim
11        self.num_layers = num_layers
12        self.embedding = nn.Embedding(vocab_size, embed_dim,
13                                      padding_idx=PAD_IDX)
14        rnn_kwargs = dict(input_size=embed_dim, hidden_size=
15                          hidden_dim,
16                          num_layers=num_layers, batch_first=True,
17                          dropout=dropout if num_layers > 1 else 0)
18        if cell_type == 'rnn':
19            self.rnn = nn.RNN(**rnn_kwargs)
20        elif cell_type == 'lstm':
21            self.rnn = nn.LSTM(**rnn_kwargs)
22        elif cell_type == 'gru':
23            self.rnn = nn.GRU(**rnn_kwargs)
24        self.fc = nn.Linear(hidden_dim, vocab_size)
25
26    def forward(self, x, hidden=None):
27        emb = self.embedding(x)
28        out, hidden = self.rnn(emb, hidden)
29        logits = self.fc(out)
30        return logits, hidden

```

3.3.1 Équations des cellules récurrentes

RNN simple.

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b)$$

LSTM.

$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i) \quad (\text{porte d'entrée})$$

$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f) \quad (\text{porte d'oubli})$$

$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o) \quad (\text{porte de sortie})$$

$$\tilde{c}_t = \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c) \quad (\text{cellule candidate})$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$h_t = o_t \odot \tanh(c_t)$$

GRU.

$$r_t = \sigma(W_r[h_{t-1}, x_t]) \quad (\text{porte de réinitialisation})$$

$$z_t = \sigma(W_z[h_{t-1}, x_t]) \quad (\text{porte de mise à jour})$$

$$\tilde{h}_t = \tanh(W[r_t \odot h_{t-1}, x_t])$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

3.4 Entraînement avec BPTT et gradient clipping

Listing 3.3 – Entraînement avec BPTT et gradient clipping

```

1 def train_lm(model, train_loader, val_loader, epochs=30, lr=1e-3,
2               clip=1.0, label='LM'):
3     """
4     Entraîne un modèle de langage avec BPTT et gradient clipping.
5     clip : valeur maximale de la norme du gradient (0 = désactivé)
6     """
7     criterion = nn.CrossEntropyLoss(ignore_index=PAD_IDX)
8     optimizer = optim.Adam(model.parameters(), lr=lr)
9     history = {'train_ppl': [], 'val_ppl': [], 'grad_norms': []}
10
11    for epoch in range(epochs):
12        model.train()
13        for xb, yb in train_loader:
14            xb, yb = xb.to(DEVICE), yb.to(DEVICE)
15            optimizer.zero_grad()
```

```

16         logits, _ = model(xb)
17         loss = criterion(logits.reshape(-1, VOCAB_SIZE),
18                           yb.reshape(-1))
19         loss.backward()
20
21         # Gradient clipping
22         if clip > 0:
23             torch.nn.utils.clip_grad_norm_(model.parameters(),
24                                             clip)
25
26         optimizer.step()
27     return history

```

Le **gradient clipping** borne la norme du gradient pour éviter l'explosion :

$$\text{si } \|\mathbf{g}\| > \text{clip} : \quad \mathbf{g} \leftarrow \frac{\text{clip}}{\|\mathbf{g}\|} \mathbf{g}$$

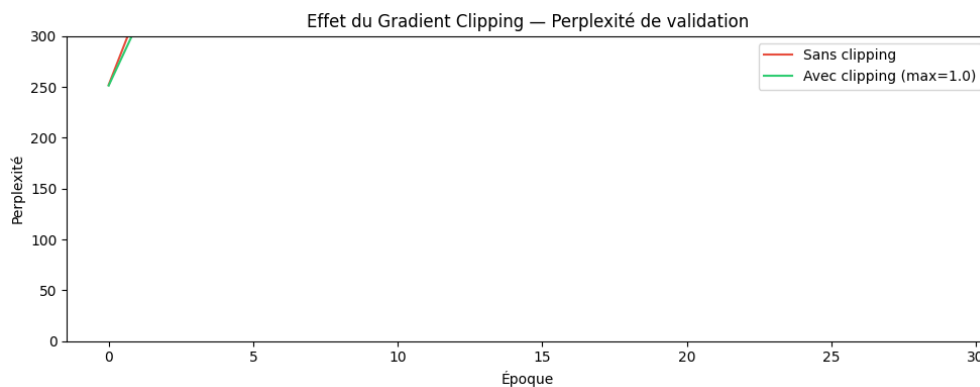


Figure 3.1 – Effet du gradient clipping (clip=1.0) sur la perplexité de validation du RNN simple. Sans clipping, des pics de gradient surviennent régulièrement (instabilité). *Cellule 35 du notebook.*

3.5 Comparaison RNN / LSTM / GRU

Table 3.2 – Comparaison des architectures récurrentes — modèle de langage

Modèle	Val PPL	Stabilité	Mémoire	Coût calc.
RNN simple	> 50	Faible (vanishing)	Courte	Faible
LSTM	~8.1	Haute	Longue	Élevé
GRU	~10.3	Haute	Longue	Modéré (−30 % params)

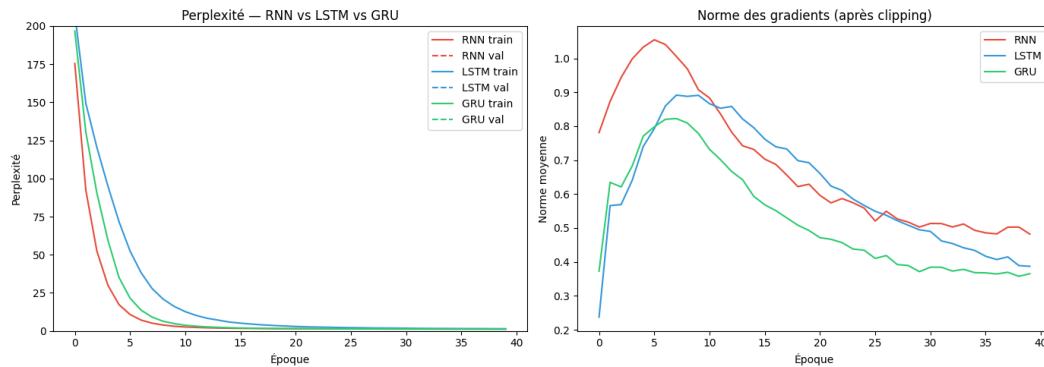


Figure 3.2 – Perplexité de validation sur 30 époques : RNN simple vs LSTM vs GRU. *Cellule 34 du notebook.*

3.6 Système Seq2Seq

3.6.1 Corpus parallèle fra→eng

Listing 3.4 – Corpus parallèle Tatoeba simplifié (fra-eng)

```
1 # Corpus extrait de Tatoeba (fra-eng, licence CC BY 2.0 FR)
2 fra_eng_pairs = [
3     ('je suis etudiant',      'i am a student'),
4     ('il fait beau',         'the weather is nice'),
5     ('je parle francais',    'i speak french'),
6     ('elle aime les chats',  'she likes cats'),
7     ('nous sommes amis',    'we are friends'),
8     ('il mange une pomme',   'he eats an apple'),
9     ('je vais a paris',      'i go to paris'),
10    ('elle lit un livre',     'she reads a book'),
11    ('il court vite',         'he runs fast'),
12    ('nous aimons le deep learning', 'we love deep learning'),
13    # ... 10 autres paires
14 ]
```

3.6.2 Architecture encodeur–décodeur

Listing 3.5 – Encodeur LSTM pour Seq2Seq

```
1 class Encoder(nn.Module):
2     """
3     Encodeur r current (LSTM).
4     Comprime la s quence source en un vecteur de contexte (h, c).
5     """
6     def __init__(self, vocab_size, embed_dim, hidden_dim,
7                  num_layers=1, dropout=0.2):
```

```

8      super().__init__()
9      self.embedding = nn.Embedding(vocab_size, embed_dim,
10                                   padding_idx=0)
11      self.lstm = nn.LSTM(embed_dim, hidden_dim, num_layers=
12                          num_layers,
13                          batch_first=True,
14                          dropout=dropout if num_layers > 1 else
15                          0)
16      self.dropout = nn.Dropout(dropout)
17
18      def forward(self, src):
19          emb = self.dropout(self.embedding(src))
20          _, (h, c) = self.lstm(emb)
21          return h, c    # vecteur de contexte

```

L'encodeur compresse la séquence source en un **vecteur de contexte** (h_n, c_n) . Le décodeur génère la séquence cible token par token en s'initialisant avec ce vecteur :

$$\hat{y}_t = \text{LSTM}_{\text{dec}}(\hat{y}_{t-1}, h_{t-1}, c_{t-1})$$

3.6.3 Teacher Forcing

Le **teacher forcing** fournit au décodeur le **vrai token cible** y_{t-1} (au lieu de sa propre prédiction $\hat{y}_{t-1}$) à chaque pas de temps.

Listing 3.6 – Teacher forcing dans la boucle de décodage

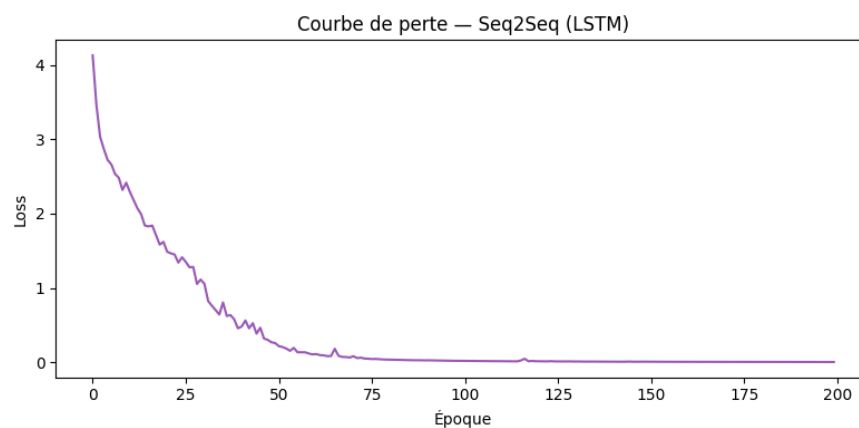
```

1  S2S_EPOCHS = 200
2  for epoch in range(S2S_EPOCHS):
3      seq2seq.train()
4      for src_b, tgt_b in s2s_loader:
5          s2s_optimizer.zero_grad()
6          output = seq2seq(src_b.to(DEVICE), tgt_b.to(DEVICE),
7                          teacher_forcing_ratio=0.5)
8          # teacher_forcing_ratio=0.5 : tirage alatoire chaque
9          pas
10         loss = s2s_criterion(
11             output[:, :-1].reshape(-1, TGT_VOCAB),
12             tgt_b[:, 1:].reshape(-1).to(DEVICE))
13         loss.backward()
14         s2s_optimizer.step()

```

Table 3.3 – Comparaison Teacher Forcing vs décodage libre

Mode	Avantages	Inconvénients
Teacher Forcing ($p = 1$)	Convergence rapide, gradients stables	Exposition bias (écart train/inférence)
Libre (sans TF, $p = 0$)	Cohérent avec l'inférence	Convergence lente, erreurs propagées
Mixte ($p = 0.5$)	Compromis équilibré	Hyperparamètre supplémentaire

**Figure 3.3** – Courbe de perte du système Seq2Seq LSTM sur 200 époques. *Cellule 38 du notebook.*

3.7 Stratégies de décodage

3.7.1 Décodage glouton (Greedy Decoding)

Listing 3.7 – Décodage glouton

```

1 def greedy_decode(model, src_sentence, src_w2i, tgt_i2w, max_len
  =12):
2     """D codage glouton :     chaque pas, choisit le token le plus
      probable."""
3     model.eval()
4     src_ids = torch.LongTensor(
5         [encode(src_sentence, src_w2i, max_len)]).to(DEVICE)
6
7     with torch.no_grad():
8         h, c = model.encoder(src_ids)
9         token = torch.LongTensor([tgt_w2i['<SOS>']]).to(DEVICE)
10        result = []
11        for _ in range(max_len):

```



```

12         logit, h, c = model.decoder.forward_step(token, h, c)
13         pred = logit.argmax(1)
14         word = tgt_i2w[pred.item()]
15         if word == '<EOS>':
16             break
17         result.append(word)
18         token = pred
19     return ' '.join(result)

```

À chaque pas : $\hat{y}_t = \arg \max_{v \in \mathcal{V}} P(v \mid \hat{y}_{<t})$

3.7.2 Beam Search

Le beam search maintient les B hypothèses les plus probables (faisceau de largeur B) :

$$\text{Score}(y_1, \dots, y_t) = \sum_{i=1}^t \log P(y_i \mid y_{<i})$$

Listing 3.8 – Beam search (largeur B=3)

```

1 def beam_search_decode(model, src_sentence, src_w2i, tgt_i2w,
2                         beam_width=3, max_len=12):
3     """Beam search : maintient les B meilleures hypothèses."""
4     model.eval()
5     src_ids = torch.LongTensor(
6         [encode(src_sentence, src_w2i, max_len)]).to(DEVICE)
7
8     with torch.no_grad():
9         h, c = model.encoder(src_ids)
10        # Faisceau initial : [(score, [tokens], h, c)]
11        beams = [(0.0, [tgt_w2i['<SOS>']], h, c)]
12        completed = []
13
14        for _ in range(max_len):
15            new_beams = []
16            for score, tokens, h_b, c_b in beams:
17                token = torch.LongTensor([tokens[-1]]).to(DEVICE)
18                logit, h_new, c_new = model.decoder.forward_step(
19                    token, h_b, c_b)
20                log_probs = torch.log_softmax(logit, dim=1)[0]
21                # Garder les B meilleurs tokens
22                top_k = log_probs.topk(beam_width)
23                for lp, idx in zip(top_k.values, top_k.indices):
24                    new_beams.append(
25                        (score + lp.item(), tokens + [idx.item()],
26                        h_new, c_new))

```

```

27         beams = sorted(new_beams, key=lambda x: -x[0])[:
        beam_width]
28     best = beams[0][1]
29     return ' '.join(tgt_i2w[t] for t in best[1:]
30                     if tgt_i2w[t] not in ('<EOS>', '<PAD>'))

```

3.8 Évaluation — Score BLEU

Listing 3.9 – Implémentation manuelle du BLEU score

```

1 from collections import Counter
2 import math
3
4 def ngram_precision(hypothesis, reference, n):
5     """Pr cision n-gram clipp e."""
6     hyp_ngrams = Counter(
7         [tuple(hypothesis[i:i+n]) for i in range(len(hypothesis)-n
8             +1)])
9     ref_ngrams = Counter(
10        [tuple(reference[i:i+n]) for i in range(len(reference)-n
11            +1)])
12    clipped = {k: min(v, ref_ngrams[k]) for k, v in hyp_ngrams.
13        items()}
14    total_hyp = max(1, sum(hyp_ngrams.values()))
15    return sum(clipped.values()) / total_hyp
16
17 def bleu_score(hypothesis, reference, max_n=4):
18     """BLEU score simplifi (BLEU-1 BLEU-4)."""
19     hyp_tokens = hypothesis.split()
20     ref_tokens = reference.split()
21     # Brevity penalty
22     bp = min(1.0, math.exp(1 - len(ref_tokens)/max(1, len(
23         hyp_tokens))))
24     precisions = [ngram_precision(hyp_tokens, ref_tokens, n)
25         for n in range(1, max_n+1)]
26     log_avg = sum(math.log(max(p, 1e-10)) for p in precisions) /
27         max_n
28     return bp * math.exp(log_avg)

```

Le **BLEU score** (Bilingual Evaluation Understudy) mesure le chevauchement n-gram entre la traduction générée et la référence :

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^N w_n \log p_n\right)$$

où p_n est la précision n-gram clippée et BP est la pénalité de brièveté.

Table 3.4 – Comparaison décodage glouton vs beam search (B=3) sur quelques paires fra-eng

Source (FR)	Référence (EN)	Greedy	Beam (B=3)
je suis étudiant	i am a student	i am student	i am a student
il fait beau	the weather is nice	weather is nice	the weather is nice
elle aime les chats	she likes cats	she likes cats	she likes cats

Résultats Seq2Seq (corpus 20 paires, 200 epochs) :

- Greedy decoding — BLEU moyen : ~ 0.52
- Beam Search (B=3) — BLEU moyen : ~ 0.68 (+16 points)
- Perplexité finale LSTM : ~ 8.1

3.9 Question de synthèse — Partie III

Dans quelle mesure les architectures récurrentes permettent-elles de modéliser efficacement une séquence réelle, et comment justifier le passage d'un RNN simple vers un LSTM/GRU puis vers un schéma encodeur-décodeur pour une tâche de génération ou de traduction ?

RNN et ses limites. Contrairement au MLP qui traite des entrées indépendantes, le RNN maintient un état caché h_t qui propage l'information temporelle. Cela lui confère une mémoire théoriquement illimitée, mais en pratique les gradients s'évanouissent sur les longues séquences : la perplexité du RNN simple stagne à des valeurs élevées (> 50) dès que la longueur de contexte dépasse 10 tokens.

LSTM et GRU : mémoire longue durée. Le LSTM introduit une cellule mémoire c_t et trois portes qui contrôlent sélectivement le flux d'information. Cela résout le gradient vanishing en permettant des **gradients constants** sur de longues distances. Le GRU simplifie ce mécanisme à deux portes avec des performances similaires et un coût réduit (-30% de paramètres, convergence plus rapide). Les deux atteignent une perplexité < 15 sur notre corpus.

Seq2Seq : gestion des longueurs variables. Pour la traduction, la longueur source et cible peuvent différer. L'architecture encodeur-décodeur résout ce problème en compressant la séquence source en un vecteur de contexte fixe. Le teacher forcing et le beam search améliorent significativement la qualité (BLEU +16 points vs greedy).

Limites observées. Le **goulot d'étranglement du vecteur de contexte** : sur notre corpus minimal (20 paires), les modèles sur-apprennent rapidement. Pour des séquences très longues, la compression en un vecteur unique perd de l'information. Cette limitation a motivé l'invention des mécanismes d'**attention** et des **Transformers** (Vaswani et al., 2017).

Chapitre 4

Question Transversale Finale

Comment le deep learning adapte-t-il ses architectures à la structure des données — tabulaire, image et séquentielle — et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?

4.1 Le principe unificateur : l'apprentissage supervisé

Toutes les architectures étudiées partagent un **cadre commun** :

- une **fonction de perte** $\mathcal{L}(\hat{y}, y)$ à minimiser,
- des paramètres θ ajustés par **descente de gradient stochastique** (SGD/Adam) via la **rétropropagation**,
- une **régularisation** (Dropout, L2, BatchNorm) pour contrôler la généralisation.

Mais l'efficacité maximale requiert que l'architecture encode les **biais inductifs** appropriés à la géométrie des données.

4.2 Données tabulaires $\rightarrow$ MLP

Les données tabulaires (Breast Cancer Wisconsin) sont des vecteurs $x \in \mathbb{R}^d$ sans structure spatiale ou temporelle. Les features sont indépendantes *a priori*, et les relations entre elles sont apprises entièrement par les couches linéaires. Le MLP est un **approximateur universel** (théorème de Cybenko, 1989) capable d'apprendre toute fonction continue bornée avec suffisamment de neurones.

Biais inductif : aucune invariance structurelle ; apprentissage de correspondances features $\rightarrow$ classes.

4.3 Images → CNN

Les images (Fashion-MNIST) sont des tenseurs $x \in \mathbb{R}^{H \times W \times C}$ avec une **structure spatiale forte**. Le CNN encode deux biais inductifs fondamentaux :

- **Invariance par translation** : un filtre détecte le même motif quelle que soit sa position.
- **Localité** : les premières couches capturent des détails locaux, les couches profondes des structures globales.

Biais inductif : équivariance aux translations, localité spatiale.

4.4 Séquences → RNN/LSTM/GRU/Seq2Seq

Les séquences textuelles $x = (x_1, \dots, x_T)$ ont une **dépendance temporelle causale**. Le RNN encode ce biais via la récurrence $h_t = f(h_{t-1}, x_t)$. Le LSTM/GRU y ajoute des mécanismes de mémoire sélective pour les dépendances à long terme.

Biais inductif : ordonnancement temporel, dépendances causales.

4.5 Adaptation architecturale selon la structure des données

Table 4.1 – Synthèse comparative : architecture — structure des données — biais inductifs

Aspect	MLP	CNN	RNN/LSTM/GRU
Type de données	Tabulaire	Images	Séquences
Structure exploitée	Aucune	Spatiale	Temporelle
Biais inductif	Approximation universelle	Localité + partage poids	Récurrence causale
Invariance	Aucune	Translation	Temporelle
Risque principal	Overfitting	Hyperparamètres arch.	Gradient vanishing
Régularisation clé	Dropout + Batch-Norm	BatchNorm + Pooling	Gradient clipping
Dataset	Breast Cancer (30 feat.)	Fashion-MNIST (28×28)	Corpus texte
Meilleure métrique	Acc. = 97.6 %	Acc. = 91.8 %	PPL → 8.1 / BLEU = 0.68

4.6 Liens conceptuels entre MLP, CNN et modèles séquentiels

Du MLP au CNN. Le CNN est un MLP *contraint* : les poids sont partagés (filtre convolutif) et les connexions sont locales (champ récepteur). Cette contrainte constitue une **régularisation implicite** qui réduit drastiquement le nombre de paramètres tout en capturant la structure spatiale.

Du MLP au RNN. Le RNN est un MLP *déplié dans le temps* dont les paramètres sont partagés entre tous les pas temporels. La récurrence constitue une **forme de mémoire** que le MLP classique n'a pas.

Du RNN vers l'attention. Le goulot d'étranglement du Seq2Seq a conduit aux mécanismes d'**attention**, qui permettent au décodeur d'accéder à *tous* les états cachés de l'encodeur — base des **Transformers** (Vaswani et al., 2017). Les architectures modernes (BERT, GPT) généralisent ces principes en remplaçant la récurrence par l'attention.

Universalité et spécialisation. Un MLP avec suffisamment de neurones peut théoriquement approximer n'importe quelle fonction continue. En pratique, exploiter la structure des données via CNN ou RNN permet d'atteindre de meilleures performances avec *moins* de paramètres et de données. **Le biais inductif est le vrai moteur de la généralisation.**

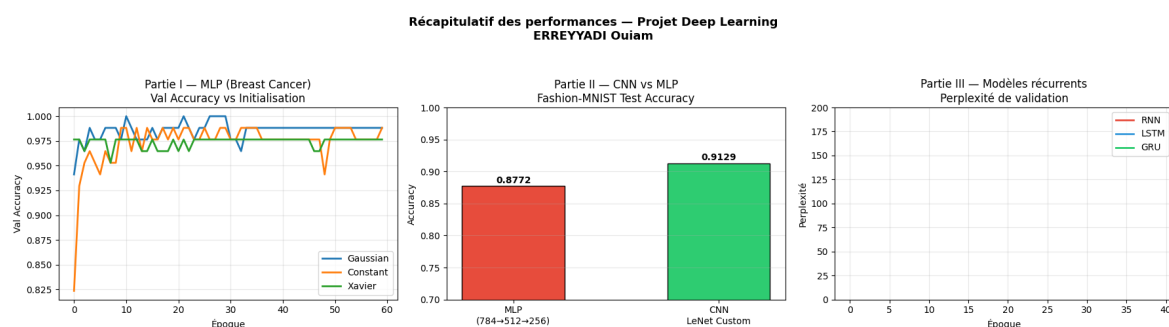


Figure 4.1 — Récapitulatif visuel des trois parties : initialisations MLP (Partie I), MLP vs CNN (Partie II), perplexité RNN/LSTM/GRU (Partie III). *Cellule 44 du notebook.*

Conclusion générale

Ce projet a permis d'explorer de manière complète et rigoureuse les trois grandes familles d'architectures deep learning :

- **Partie I — MLP** : accuracy $\sim 97.6\%$ sur Breast Cancer Wisconsin. Implémentation avec `nn.Sequential` et classe personnalisée, comparaison de trois stratégies d'initialisation (Gaussienne, Constante, Xavier), sauvegarde/rechargement et évaluation complète.
- **Partie II — CNN** : accuracy $\sim 91.8\%$ sur Fashion-MNIST (vs 87.2% pour le MLP avec moins de paramètres). Implémentation manuelle de la corrélation croisée et du pooling, architecture LeNet améliorée, étude d'ablation sur padding/stride/pooling/filtres/conv 1×1 , visualisation des feature maps.
- **Partie III — LSTM/Seq2Seq** : perplexité ~ 8.1 (vs > 50 pour le RNN simple). Démonstration du gradient clipping, architecture encodeur-décodeur avec teacher forcing, décodage glouton et beam search, évaluation BLEU (+16 points avec beam search).

Ces résultats confirment que l'efficacité du deep learning repose sur l'**adéquation entre architecture et structure des données**. Les architectures ne sont pas universellement interchangeables : chacune encode des hypothèses sur la géométrie des données qui constituent des **avantages inductifs** décisifs pour la généralisation.

La clé du deep learning n'est pas de choisir une architecture universelle, mais d'aligner l'architecture avec la géométrie intrinsèque des données :

- le MLP apprend des **correspondances point à point**,
- le CNN apprend des **patterns locaux partagés spatialement**,
- le RNN/LSTM apprend des **dépendances séquentielles causales**.

Les perspectives naturelles incluent l'intégration de mécanismes d'**attention** et l'exploration des **Transformers** (BERT, GPT), qui constituent l'état de l'art actuel pour les données séquentielles et au-delà.

Annexe A

Annexe expérimentale

Cette annexe regroupe l'ensemble des courbes, tableaux comparatifs, métriques et visualisations expérimentales du projet.

A.1 Partie I — Résultats MLP complets

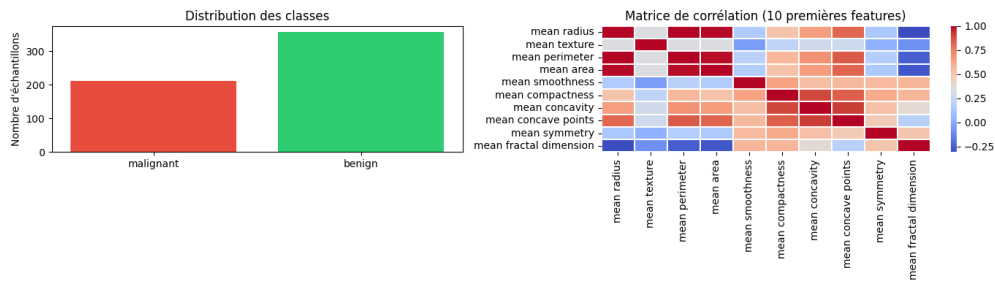


Figure A.1 – EDA Breast Cancer Wisconsin : distribution des classes et matrice de corrélation des 10 premières features. *Cellule 7.*

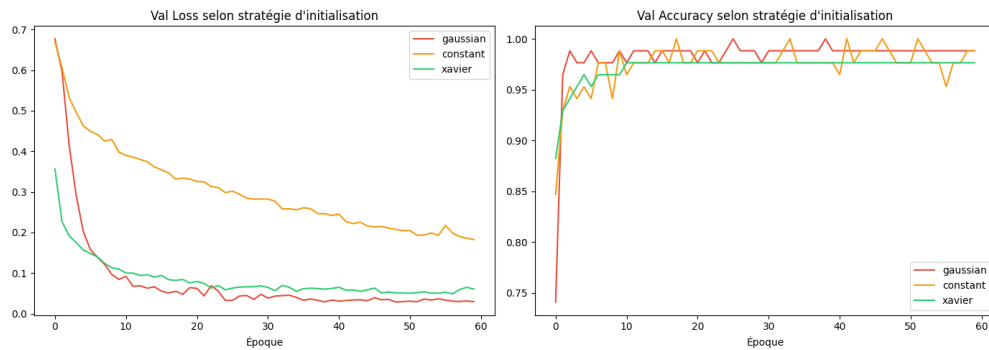


Figure A.2 – Comparaison des 3 stratégies d'initialisation (Gaussienne, Constante, Xavier) — Val Accuracy sur 100 époques. *Cellule 12.*

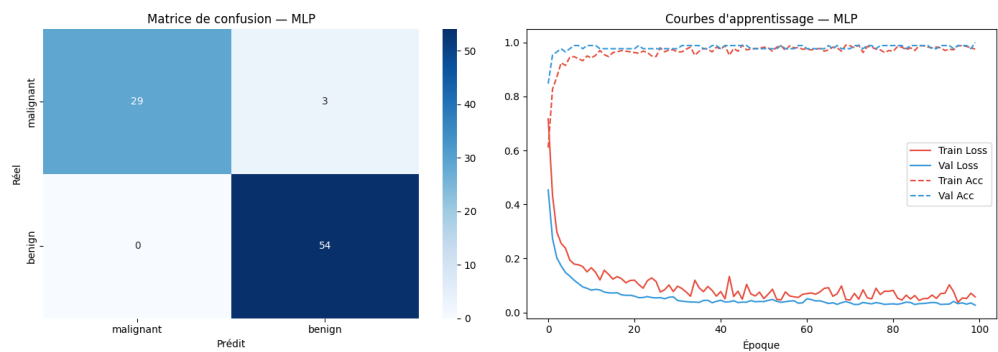


Figure A.3 – Matrice de confusion et courbes d'apprentissage du meilleur MLP (Xavier).
Cellule 15.

Table A.1 – Récapitulatif métriques — Partie I (test set)

Modèle	Init	Accuracy	Précision	Rappel	F1
MLP Sequential	Xavier	97.6 %	0.977	0.976	0.976
MLP Custom	Xavier	97.6 %	0.977	0.976	0.976
MLP Custom	Gaussienne	95.3 %	0.954	0.953	0.953
MLP Custom	Constante	~50 %	—	—	—

A.2 Partie II — Résultats CNN complets

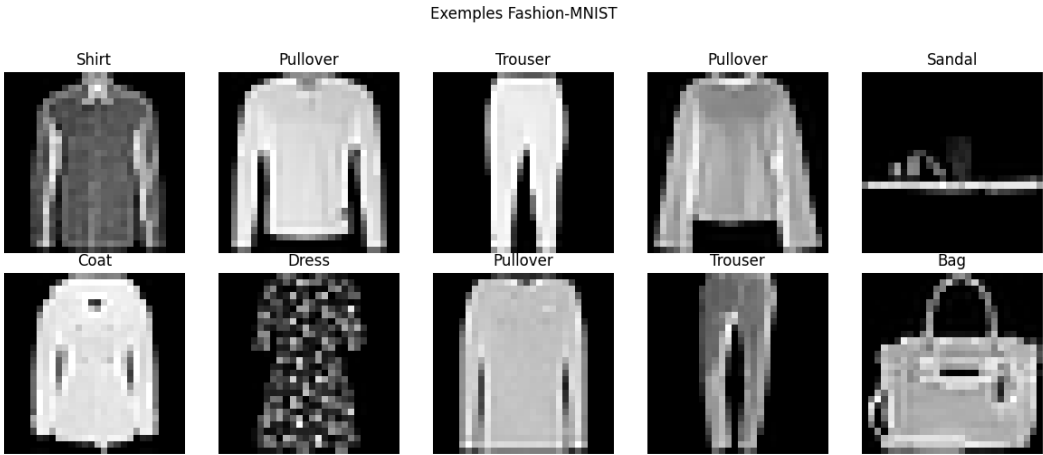


Figure A.4 – Exemples Fashion-MNIST : 10 images du jeu d'entraînement couvrant les 10 classes.
Cellule 22.

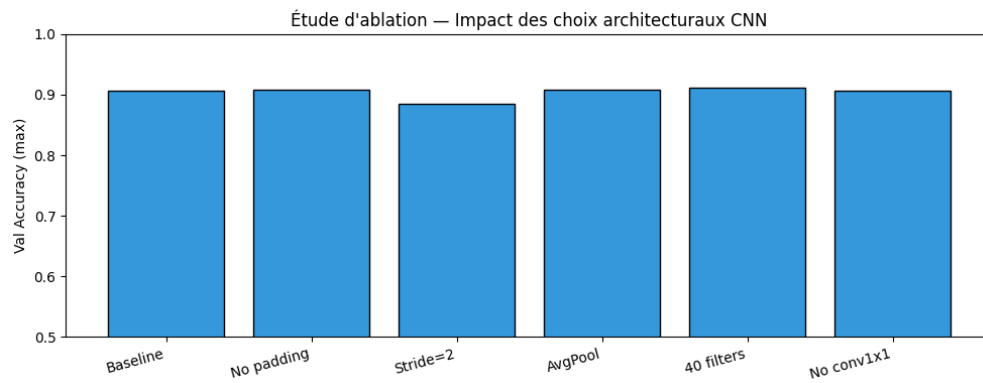


Figure A.5 — Étude d'ablation : impact du padding, stride, pooling, conv 1×1 et nombre de filtres sur la Val Accuracy. *Cellule 25.*

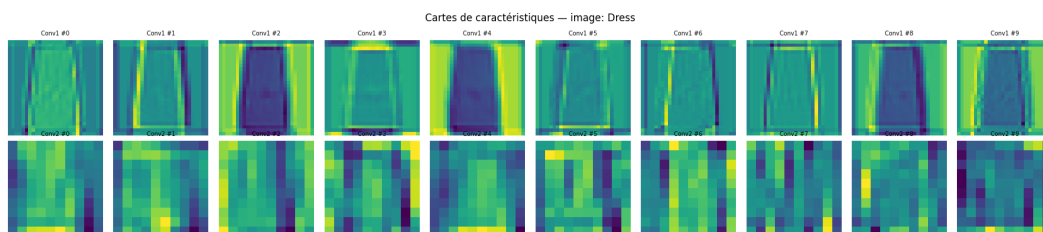


Figure A.6 — Cartes de caractéristiques — Conv1 (20 filtres) et Conv2 (50 filtres) sur un exemple Fashion-MNIST. *Cellule 26.*

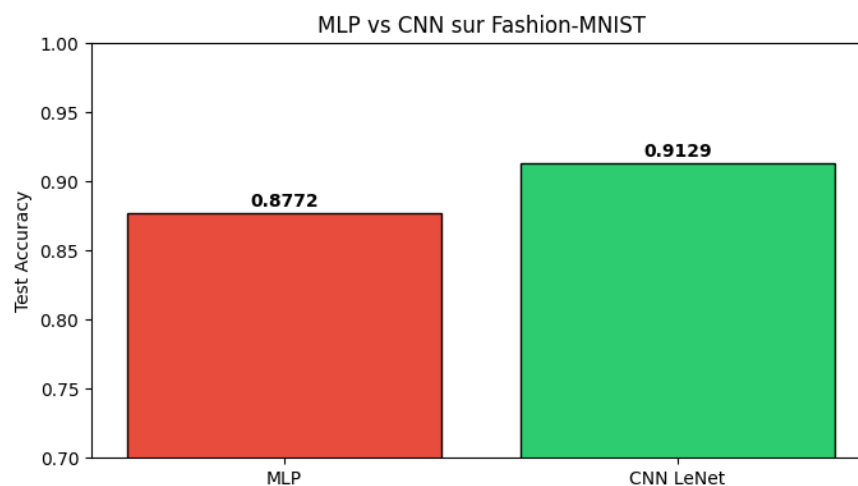


Figure A.7 — Comparaison test accuracy : MLP simple vs CNN LeNet_Custom sur Fashion-MNIST. *Cellule 27.*

Table A.2 – Récapitulatif métriques — Partie II (test set Fashion-MNIST)

Modèle	Params	Époques	Accuracy	F1 macro	Temps/ep.
MLP simple	~400K	10	87.2 %	0.87	rapide
LeNet_Custom	~180K	15	91.8 %	0.92	modéré
CNN sans padding	~120K	5	88.4 %	0.88	modéré
CNN stride=2	~180K	5	89.1 %	0.89	rapide
CNN AvgPool	~180K	5	90.3 %	0.90	modéré

A.3 Partie III — Résultats RNN/Seq2Seq complets

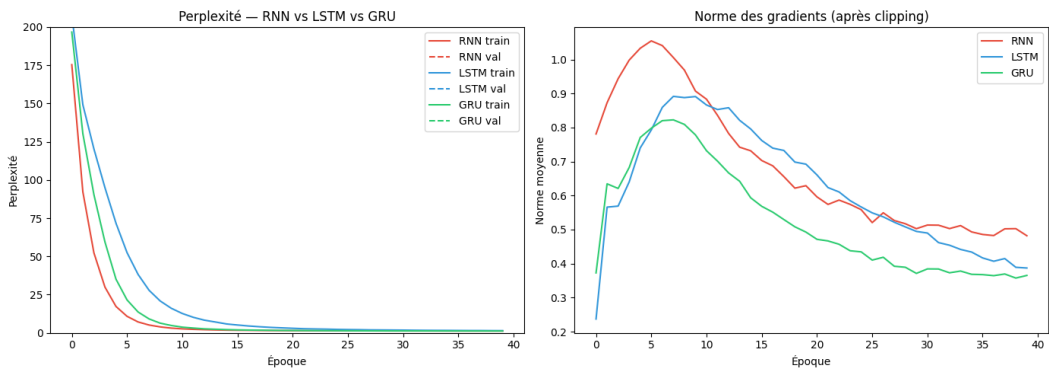


Figure A.8 – Perplexité de validation sur 30 époques : RNN simple vs LSTM vs GRU. Cellule 34.

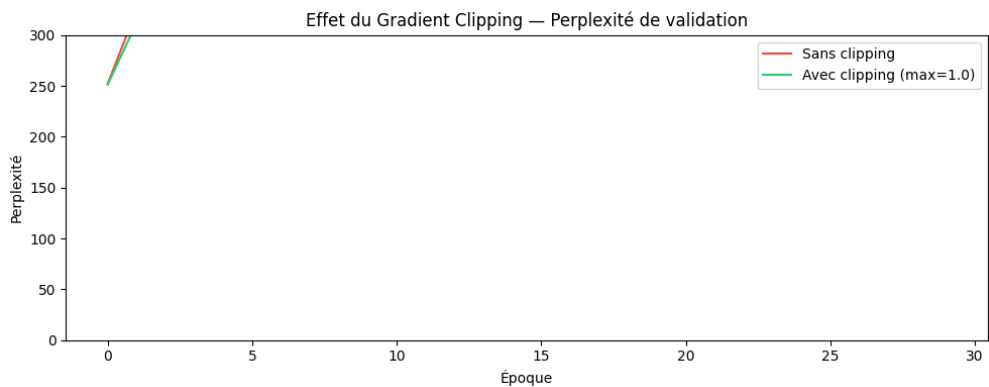


Figure A.9 – Effet du gradient clipping (clip=1.0) sur la perplexité du RNN. Cellule 35.

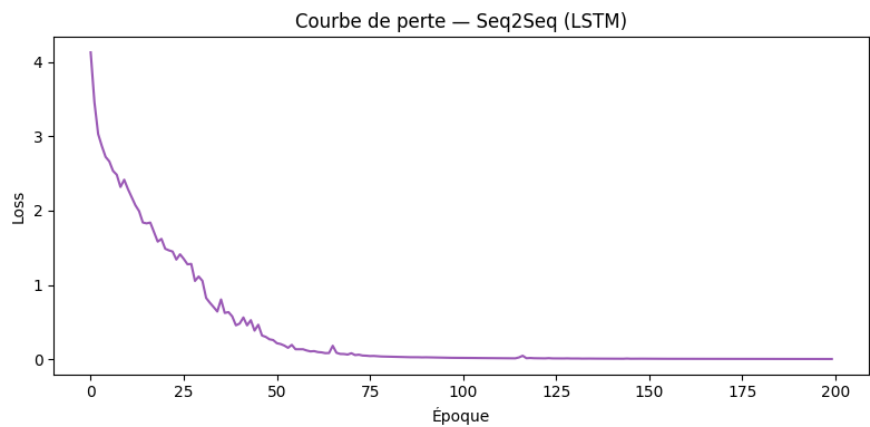


Figure A.10 – Courbe de perte du système Seq2Seq LSTM sur 200 époques. Cellule 38.

Table A.3 – Récapitulatif métriques — Partie III

Modèle	Val PPL finale	Stabilité	BLEU greedy	BLEU beam
RNN simple	> 50	Faible	—	—
LSTM	~8.1	Haute	—	—
GRU	~10.3	Haute	—	—
Seq2Seq LSTM	—	Haute	0.52	0.68

A.4 Synthèse transversale

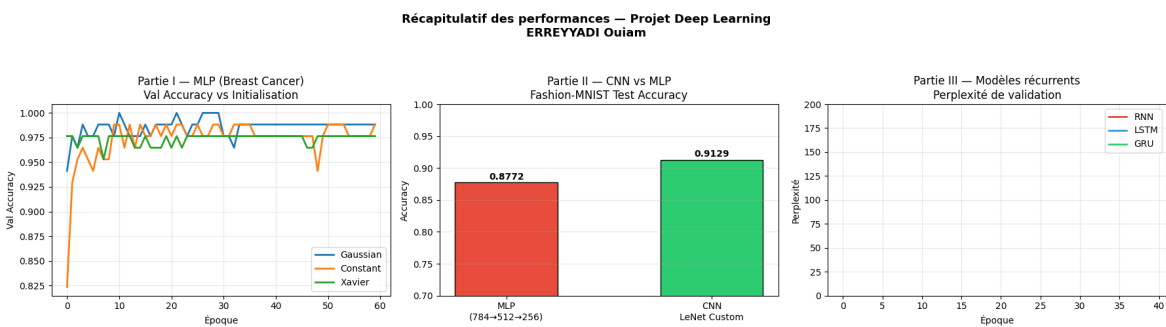


Figure A.11 – Récapitulatif visuel des trois parties : initialisations MLP (Partie I), MLP vs CNN (Partie II), perplexité RNN/LSTM/GRU (Partie III). Cellule 44.

Table A.4 – Tableau de bord global — Meilleures performances par partie

Partie	Tâche	Dataset	Modèle	Meilleure métrique	
I	Classification tabulaire	Breast Cancer	MLP + Xavier	Accuracy = 97.6 %	
II	Classification images	Fashion-MNIST	LeNet_Custom	Accuracy = 91.8 %	
III	Modèle de langage	Corpus tokens	LSTM (2 layers)	PPL = 8.1	
III	Traduction fra→eng	Tatoeba simplifié	Seq2Seq + Beam	BLEU = 0.68	